

Redis Event Schema

Redis Event Schema

Overview

This document defines the **canonical Redis event schema** for your system. It is the single source of truth for:

- Gateway  Agent Harness communication
- Frontend SSE event streaming
- Workflow engine subscriptions

All events are JSON objects published over Redis Pub/Sub channels.

Folder Structure

```
/redis-schema  
eventTypes.ts  
channels.md  
payloads.md
```

File: eventTypes.ts

```
export type RedisEventType =  
  // Task lifecycle  
  | "task.created"  
  | "task.updated"  
  | "task.resume"  
  | "task.failed"  
  | "task.completed"  
  
  // Session lifecycle  
  | "session.created"  
  | "session.stateChanged"  
  
  // Logs  
  | "log.append"  
  
  // Output streaming  
  | "output.partial"  
  | "output.final"  
  
  // Human-in-the-loop  
  | "approval.requested";  
  
export interface RedisEventBase {  
  type: RedisEventType;  
  sessionId?: string;  
  taskId?: string;  
  timestamp: number;  
}
```

File: channels.md

Redis Channels

1. Task control channel

- **Channel:** `tasks`
- **Used by:** Gateway → Agent Harness
- **Event types:**
 - `task.created`
 - `task.resume`

2. Agent log channel

- **Channel:** `agent.logs`
- **Used by:** Agent Harness → Gateway
- **Event types:**
 - `log.append`

3. Agent event channel

- **Channel:** `agent.events`
- **Used by:** Agent Harness → Gateway
- **Event types:**
 - `approval.requested`
 - `task.updated`
 - `task.failed`
 - `task.completed`

4. Session event channels

- **Channel pattern:** `session:{sessionId}:events`
- **Used by:** Agent Harness → Gateway → Frontend SSE
- **Event types:**
 - `output.partial`
 - `output.final`
 - `session.stateChanged`
 - `log.append` (optional fan-out)

File: payloads.md

Redis Event Payloads

All payloads are JSON objects.

1. `task.created`

- **Channel:** `tasks`
- **Shape:**

```
```json
{
 "type": "task.created",
 "taskId": "string",
 "sessionId": "string", // optional, if pre-bound
 "timestamp": 1710000000000
}
```

### 2. task.resume

- **Channel:** tasks

```
{
 "type": "task.resume",
 "taskId": "string",
 "sessionId": "string",
 "timestamp": 1710000000000
}
```

### 3. log.append

- **Channel:** agent.logs (and optionally session:{sessionId}:events)

```
{
 "type": "log.append",
 "sessionId": "string",
 "taskId": "string",
 "timestamp": 1710000000000,
 "payload": {
 "message": "string",
 "severity": "debug" | "info" | "warn" | "error"
 }
}
```

## 4. output.partial

- **Channel:** session:{sessionId}:events

```
{
 "type": "output.partial",
 "sessionId": "string",
 "taskId": "string",
 "timestamp": 1710000000000,
 "payload": {
 "chunk": "string"
 }
}
```

## 5. output.final

- **Channel:** session:{sessionId}:events

```
{
 "type": "output.final",
 "sessionId": "string",
 "taskId": "string",
 "timestamp": 1710000000000,
 "payload": {
 "output": {}
 }
}
```

## 6. session.stateChanged

- **Channel:** session:{sessionId}:events

```
{
 "type": "session.stateChanged",
 "sessionId": "string",
 "timestamp": 1710000000000,
 "payload": {
 "from": "idle" | "running" | "waiting" | "completed" | "failed",
 "to": "idle" | "running" | "waiting" | "completed" | "failed"
 }
}
```

## 7. approval.requested

- **Channel:** agent.events

```
{
 "type": "approval.requested",
 "taskId": "string",
 "sessionId": "string",
 "timestamp": 1710000000000,
 "payload": {
 "promptMessage": "string",
 "options": ["string"],
 "metadata": {}
 }
}
```

## 8. task.updated

- **Channel:** agent.events

```
{
 "type": "task.updated",
 "taskId": "string",
 "timestamp": 1710000000000,
 "payload": {
 "status": "PENDING" | "RUNNING" | "WAITING_FOR_HUMAN" | "QUEUED" | "COMPLETED"
 | "FAILED"
 }
}
```

## 9. task.completed

- **Channel:** agent.events

```
{
 "type": "task.completed",
 "taskId": "string",
 "timestamp": 1710000000000,
 "payload": {
 "summary": "string"
 }
}
```

## 10. task.failed

- **Channel:** agent.events

```
{
 "type": "task.failed",
 "taskId": "string",
 "timestamp": 1710000000000,
 "payload": {
 "reason": "string",
 "details": {}
 }
}
```

\*\*\*

## # Summary

This Redis Event Schema document defines:

- Canonical event types
- Channel naming conventions
- JSON payload shapes

It is designed to be shared across:

- Gateway (Fastify)
- Agent Harness (Node/Python)
- Frontend SSE client
- Workflow engine

You can drop these files into your repo as the authoritative contract for all Redis-based communication.